



第4章 串

- 第2章 线性表
- 第3章 栈和队列
- 第4章 串
- 第5章 数组和广义表

线性结构

可表示为: $(a_1, a_2, \dots, a_n)$

串是特殊的线性表，数据元素是单个字符。线性表的操作通常以“数据元素”为操作对象；串的操作主要以“子串”为操作对象。

补充：C语言中常用的串运算

调用标准库函数 `#include<string.h>`

串比较, `strcmp(char s1, char s2)`
串复制, `strcpy(char to, char from)`
串连接, `strcat(char to, char from)`
求串长, `strlen(char s)`

.....

Function Name	Function Description
strlen()	Returns the length of the string.
strcpy()	Copy one string to another.
strncpy()	Copy first n characters of one string to another.
strcat()	Concatenates two strings.
strncat()	Concatenates first n characters of one string to another.
strcmp()	Compares two strings.
strncmp()	Compares first n characters of two strings.
strchr()	Find the first occurrence of the given character in the string.
strrchr()	Finds the last occurrence of the given characters in the string.
strstr()	Find the given substring in the string.
strcspn()	Returns the span of the source string not containing any character of the given string.
strspn()	Returns the span of the source string containing only the characters of the given string.
strpbrk()	Finds the first occurrence of any of the characters of the given string in the source string.
strtok()	Split the given string into tokens based on some character as a delimiter.
strcoll()	Compares two strings that are passed.

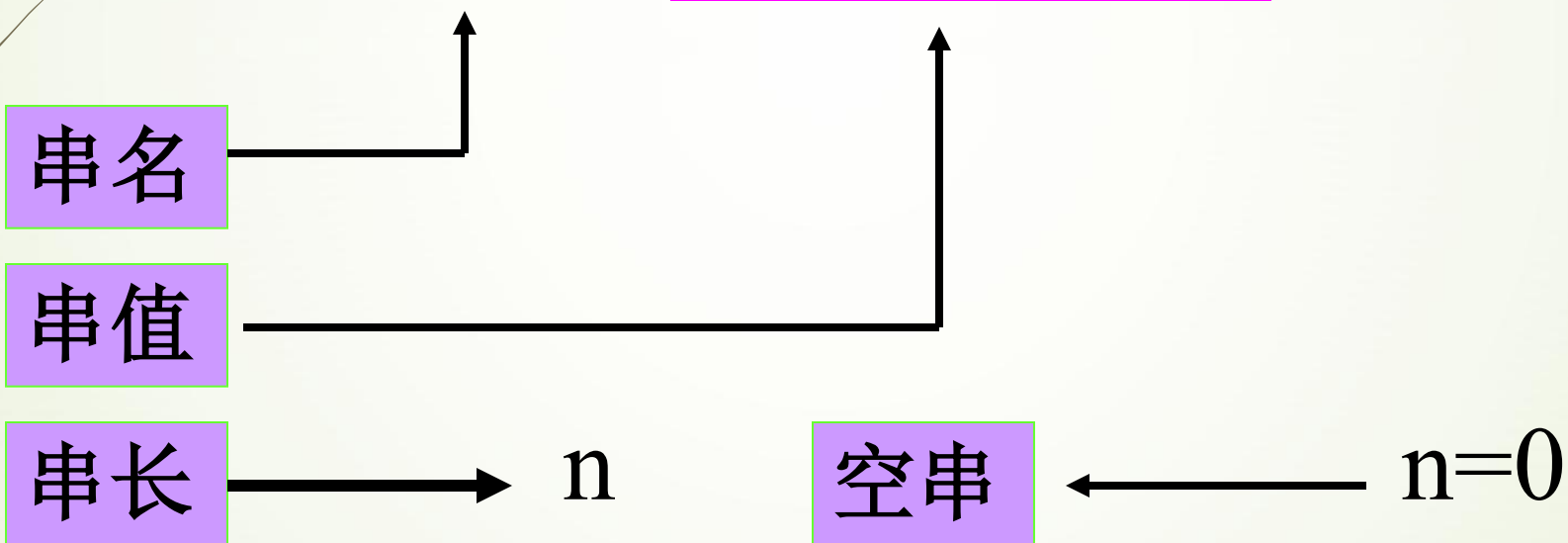
Function Name	Function Description
memset()	Initialize a block of memory with the given character.
memcmp()	Compares two blocks of memory.
memcpy()	Copy two blocks of memory.
memmove()	Moves two blocks of memory.
memchr()	Finds the given character in the block of memory.
memset()	Initialize a block of memory with the given character.
memcmp()	Compares two blocks of memory.

4.1.1 串的概念



串 (String) ---- 零个或多个字符组成的有限序列

$$S = 'a_1 a_2 \cdots a_n' \quad (n \geq 0)$$



子串：串中任意个连续的字符组成的子序列称为该串的 **子串**。

主串：包含子串的串称为 **主串**。

串相等：两个串长度相等，且对应位置的字符都相等。

空格串/空白串：由一个或多个空格组成，如 ‘ ’。

空串： **空串** 不包含任何字符，表示为 ϕ ，长度为0。

a=‘BEI‘

b=‘JING‘

c=‘BEIJING‘

d=‘BEI JING‘

4.1.2 串的类型定义、存储结构及运算



ADT String {

数据对象:

$$D = \{a_i \mid a_i \in \text{CharacterSet}, i = 1, 2, \dots, n, n \geq 0\}$$

数据关系:

$$R_1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 1, 2, \dots, n \}$$

基本操作:

(1) StrAssign (&T,chars)

//串赋值

(2) StrCompare (S,T)

//串比较

(3) StrLength (S)

//求串长

(4) Concat(&T,S1,S2)

//串联

(5) SubString(&Sub,S,**pos**,len) //求子串
(6) StrCopy(&T,S) //串拷贝
(7) StrEmpty(S) //串判空
(8) ClearString (&S) //清空串
(9) **Index(S,T,pos)** //子串的位置
(11) Replace(&S,T,V) //串替换
(12) StrInsert(&S,pos,T) //子串插入
(12) StrDelete(&S,pos,len) //子串删除
(13) DestroyString(&S) //串销毁

}ADT String

串的常见应用

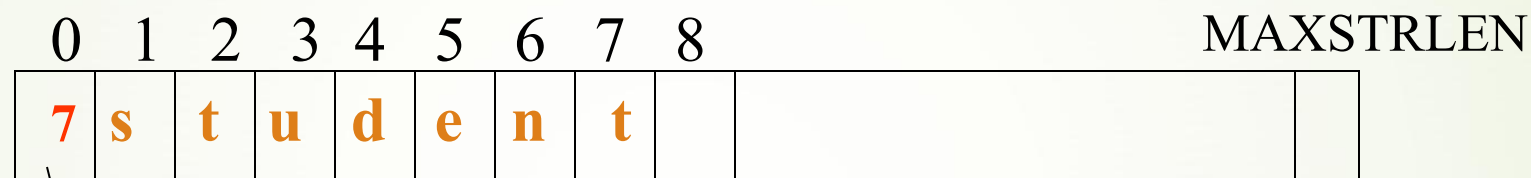
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe.config

Q1: 取路径

Q2: 取文件名

Q3: 取文件扩展名

(1) 定长顺序存储表示



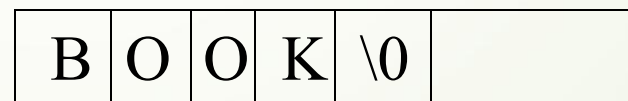
存放串的长度

约定：串值长度上溢时，用“截尾法”处理，即“截断”超过予定义长度的部分。

[存储定义]

```
#define MAXSTRLEN 255    //最大串长
typedef unsigned char SString[MAXSTRLEN+1];
```

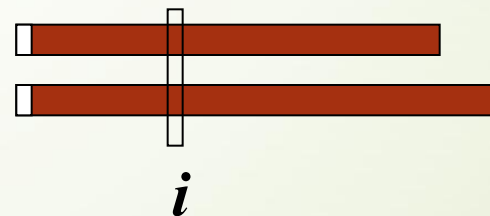
C语言本身的串表示方式:



不便于求串长等操作

[基本操作实现示例]

```
int StrCompare(SString S, SString T)
//S>T,返回值>0; S=T, 返回0; S<T, 返回值< 0
{
    for(i=1; i<=S[0] && i<=T[0]; i++)
        if( S[i] != T[i] )
            return(S[i] - T[i] );
    return S[0]-T[0]
} // StrCompare
```



```
bool Concat(SString &T, SString S1, SString S2)
```

```
//用T返回串s1和s2联接而成的新串。
```

```
//若未截断, 返回TRUE, 否则返回FALSE
```

```
{
```

```
    if( S1[0]+S2[0] <= MAXSTRLEN ) {
```

```
        T[1..S1[0]] = S1[1..S1[0]];
```

```
        T[S1[0]+1..S1[0]+S2[0]] = S2[1..S2[0]];
```

```
        T[0]= S1[0]+S2[0];  uncut=TRUE;
```

```
    }
```

```
    else if (S1[0]<MAXSTRLEN) {
```

```
        T[1..S1[0]] = S1[1..S1[0]];
```

```
        T[S1[0]+1..MAXSTRLEN] = S2[1..MAXSTRLEN-S1[0]];
```

```
        T[0]= MAXSTRLEN;  uncut=FALSE;
```

```
    }
```

```
    else {
```

```
        T[0..MAXSTRLEN]= S1[0..MAXSTRLEN];
```

```
        uncut= FALSE;
```

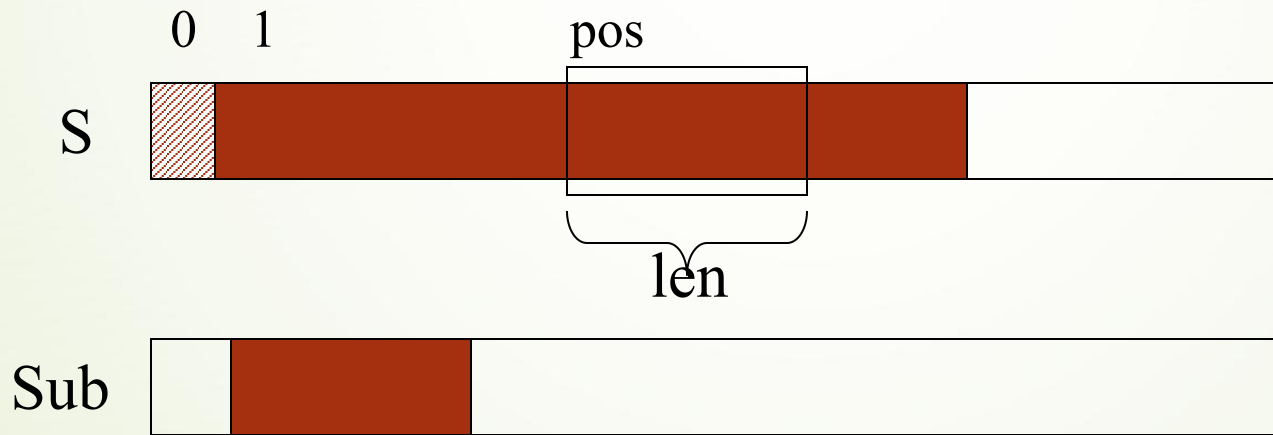
```
    }
```

```
    return uncut;
```

```
} // Concat
```



```
Status SubString(SString &Sub, SString S, int pos, int len)
//用Sub返回串S从第pos个字符起长度为len的子串
//隐含的意思: sub空间必须大于等于len+1
{
    if( pos<1 || pos>S[0] || len<0 || len>S[0]-pos+1 )
        return ERROR;
    Sub[1..len]= S[pos..pos+len-1];
    Sub[0]= len;
    return OK;
} // SubString
```



(2) 堆分配存储表示

动态分配串值存储空间，避免定长结构的截断现象。

[存储定义]

```
typedef struct {  
    char *ch;    //串空间基址，按串长申请  
    int  length; //串长度  
}HString;
```

[基本操作实现示例]

```
int StrCompare(HString S, HString T)
//S>T,返回值>0; S=T, 返回0; S<T, 返回值< 0
{
    for( i=0; i<S.length && i<T.length; i++)
        if( S.ch[i] != T.ch[i] )
            return S.ch[i] - T.ch[i];
    return S.length-T.length ;
} // StrCompare
```

S1 

S2 

```
Status Concat(HString &T, HString S1, HString S2)
```

```
//返回串S1和S2联接而成的新串T, 没有截断一说了
```

```
{
```

```
    if(T.ch) free(T.ch);    //释放T原有空间
```

```
    if(!(T.ch=(char *)malloc((S1.length+S2.length)*sizeof(char))))
```

```
        return OVERFLOW;
```

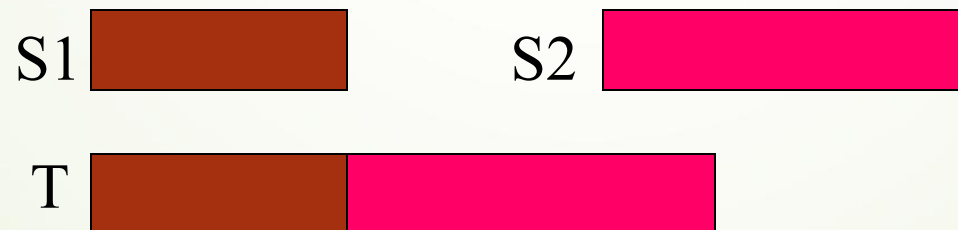
```
    T.length=S1.length+S2.length;
```

```
    T.ch[0..s1.length-1]=S1.ch[0..s1.length-1];
```

```
    T.ch[S1.length.. T.length-1]=S2.ch[0..S2.length-1];
```

```
    return OK;
```

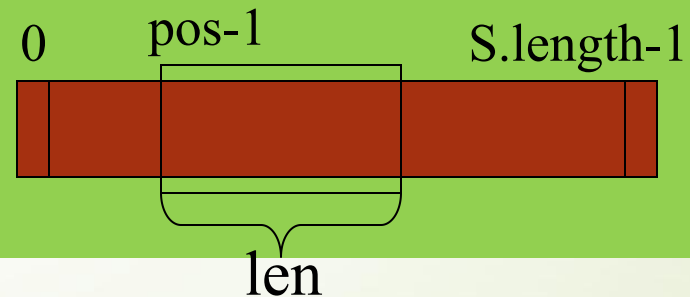
```
} // Concat
```



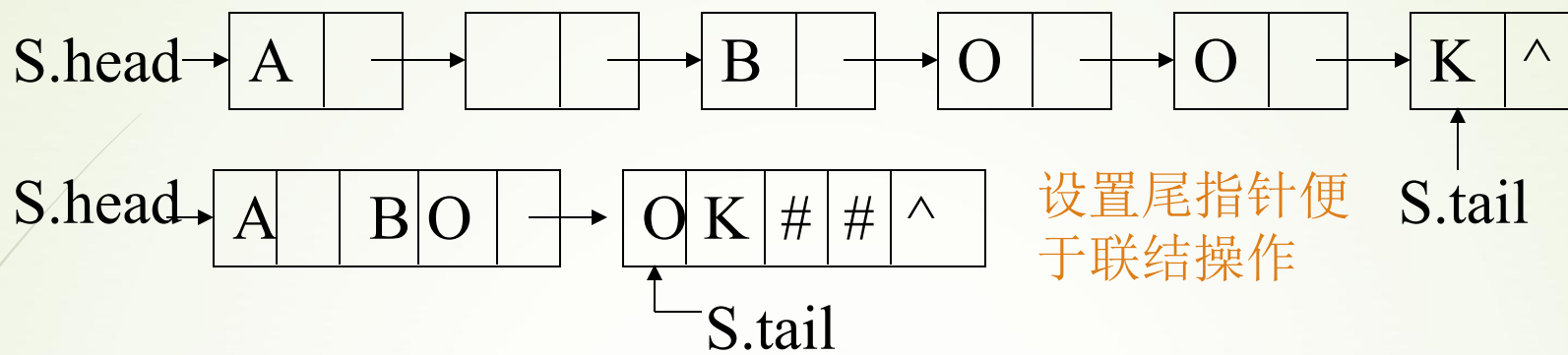
```

Status SubString(HString &Sub, HString S, int pos, int len)
//求串S从第pos个字符起长度为len的子串Sub
{
    if( pos<1 || pos >S.length || len<0 || len>S.length-pos+1 )
        return ERROR;
    if( Sub.ch)    free(Sub.ch);
if( !len ) { Sub.ch=NULL; Sub.length=0; }
    else{
        if( !(Sub.ch=(char *)malloc(len* sizeof(char)))
            return OVERFLOW;
        Sub.ch[0..len-1]=S.ch[pos-1..pos+len-2];
        Sub.length=len;
    }
    return OK;
} // SubString

```



4.2.2 串的块链存储结构



```
#define CHUNKSIZE 4 //由用户定义块大小
typedef struct Chunk {
    char ch[CHUNKSIZE];
    struct Chunk * next;
}Chunk;
typedef struct { //头尾指针的单链表，结点的data是字符块
    Chunk * head, * tail; //串的、尾头指针
    int curlen; //串当前长度
}LString;
```

占用存储量大且操作不便，实际应用很少。

串的模式匹配算法

算法目的:

确定主串中所含子串第一次出现的位置（定位）

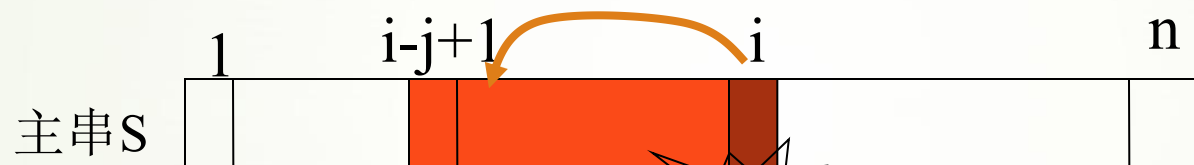
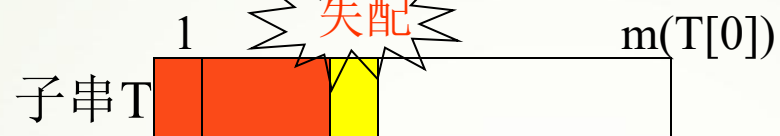
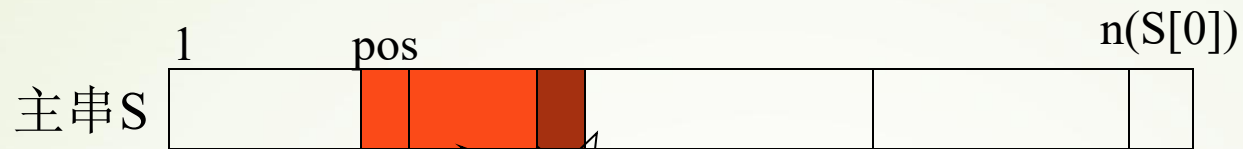
即如何实现教材 $\text{Index}(S, T, \text{pos})$ 函数

其中：S—主串，T—子串/模式串，pos—从该位序字符开始
向后寻找匹配

算法种类：以定长顺序存储结构讨论算法实现。

- **BF算法**（Brute Force，又称古典的、经典的、朴素的、穷举的）
- **KMP算法**（特点：速度快）

4.3.1 朴素的模式匹配算法



$i = \text{pos};$

$j = 1;$

定义一个变量pos
记录下匹配开始的位置

$i = \text{pos} + 1;$

$j = 1;$

在i位置失配时
利用pos将i移动一个位置
j从头开始

回溯：

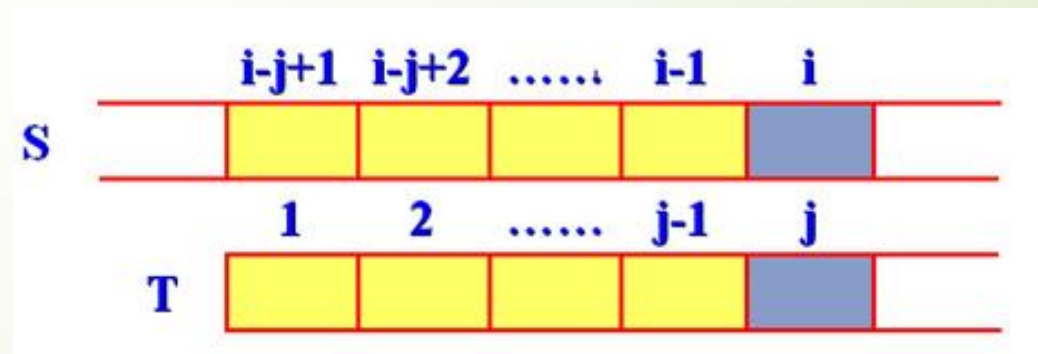
$i = i - j + 2;$

$j = 1;$

pos的值是可以计算出的
pos是 $i - j + 1$

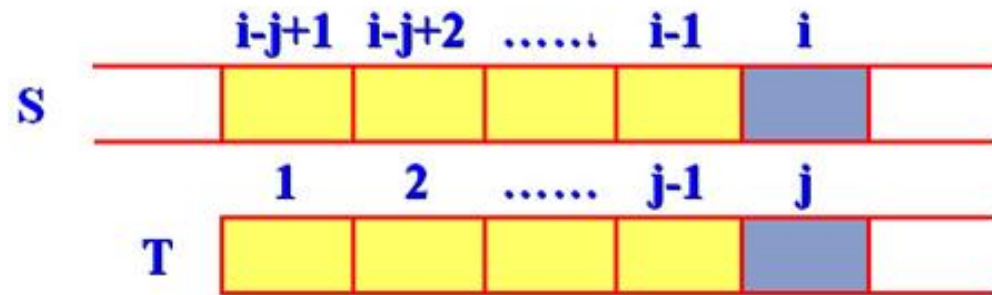
Index(S,T,pos)

- 将主串的第pos个字符(用*i*跟踪)和模式的第一个字符(用*j*跟踪)比较,
若相等,继续逐个比较后续字符($i++;j++$);
若不等,从主串的下一字符起($i=i-j+2$),重新与模式的第一个字符($j=1$)比较。
- 直到主串的一个连续子串字符序列与模式相等。返回值为S中与T匹配的子序列第一个字符的序号,即匹配成功。
- 否则,匹配失败,返回值 0



BF算法

```
int Index(SString S, SString T, int pos)
{
    int j=1, i=pos;
    while( i <= S[0] )
    {
        j = 1;
        while( j <= T[0] )
        {
            if ( S[ i ] != T[ j ] ) { ++i; ++j; }
            else { i=i-j+2; break; } //i回溯, j回到1
        }
        if( j > T[0] ) return i - T[0];
    }
    return 0;
}
```



外层循环只做了 $j=1$ 的操作；可以放到内层循环里面的else里面

BF算法

```
int Index(SString S, SString T, int pos)
{
    int j=1, i=pos;
    while( i <= S[0]  && j <= T[0])
    {
        if( S[ i ]==T[ j ]) {++i;  ++j; }
        else { i=i-j+2; j=1;} //i回溯, j回到1
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}
```



BF算法时间复杂度

例： S='0000000001', T='0001', pos=1

若 n 为主串长度， m 为子串长度，最坏情况是

- ✓ 主串前面 $n-m$ 个位置都部分匹配到子串的最后一位，即这 $n-m$ 位各比较了 m 次
- ✓ 最后 m 位也各比较了1次

总次数为： $(n-m)*m+m=(n-m+1)*m$
若 $m \ll n$ ，则算法复杂度 $O(n*m)$

KMP (Knuth Morris Pratt) 算法

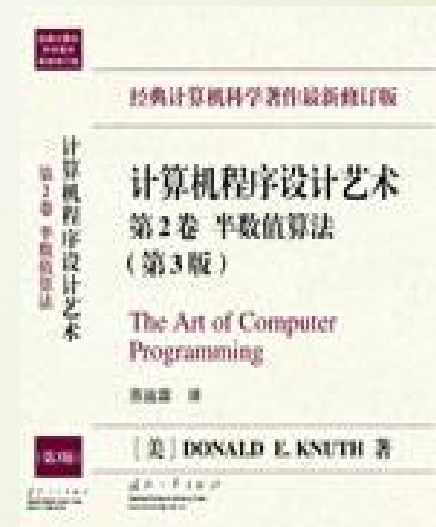
《计算机程序设计艺术 第1卷 基本算法》

《计算机程序设计艺术 第2卷 半数值算法》

《计算机程序设计艺术 第3卷 排序与查找》

...

<http://www-cs-faculty.stanford.edu/~knuth/>



KMP算法设计思想

利用已经部分匹配的结果而加快模式串的滑动速度？
且主串S的指针i不必回溯！可提速到 $O(n+m)$ ！

比较次数:1

[illegible]

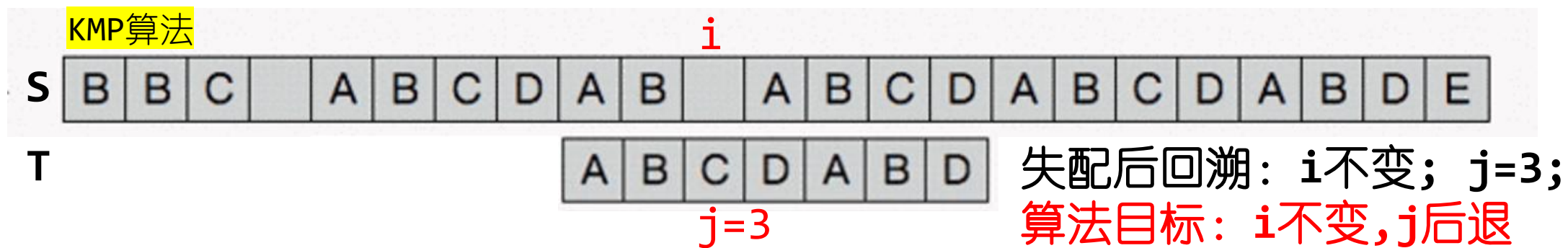
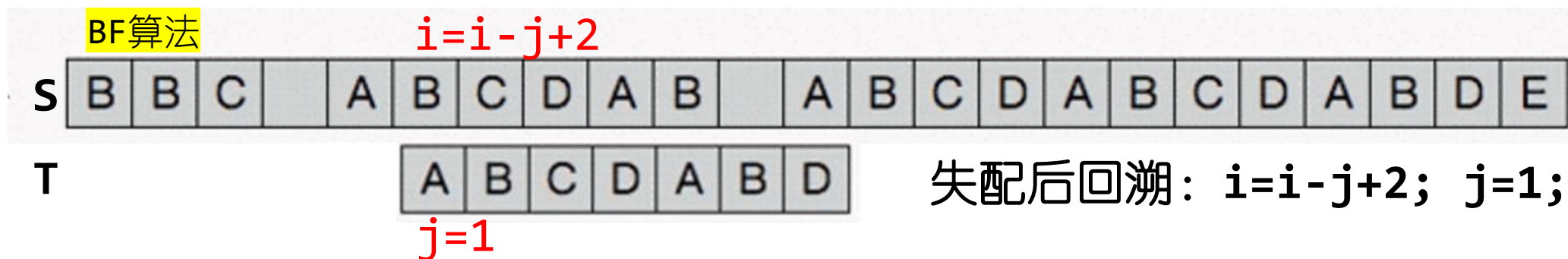
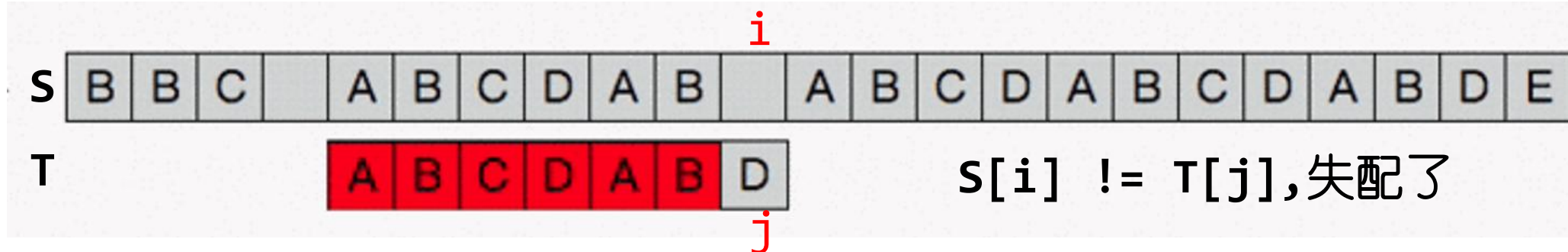
比较次数:1

[illegible]

KMP算法设计思想

利用已经部分匹配的结果而加快模式串的滑动速度？

且主串S的指针*i*不必回溯！可提速到 $O(n+m)$ ！

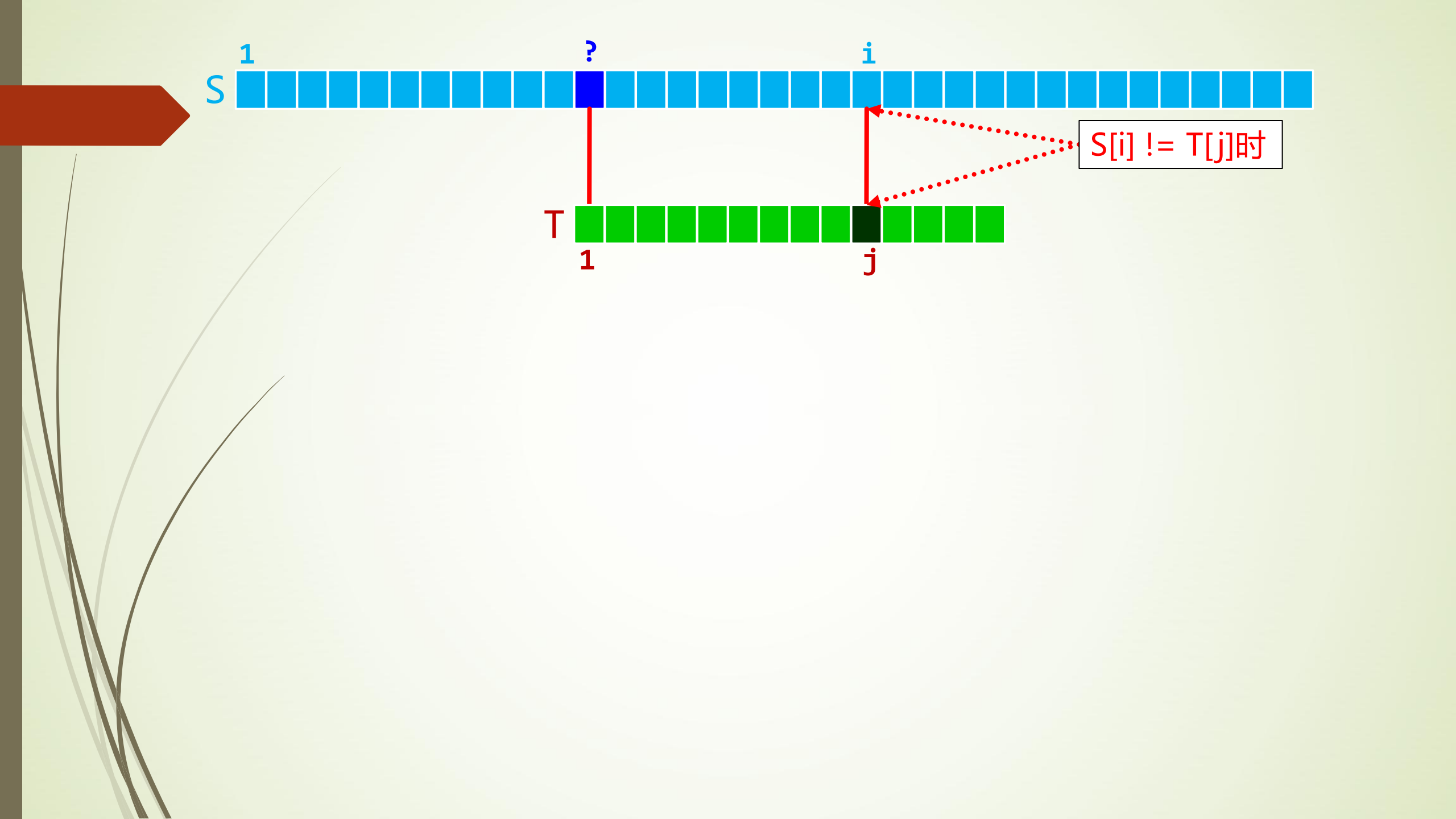


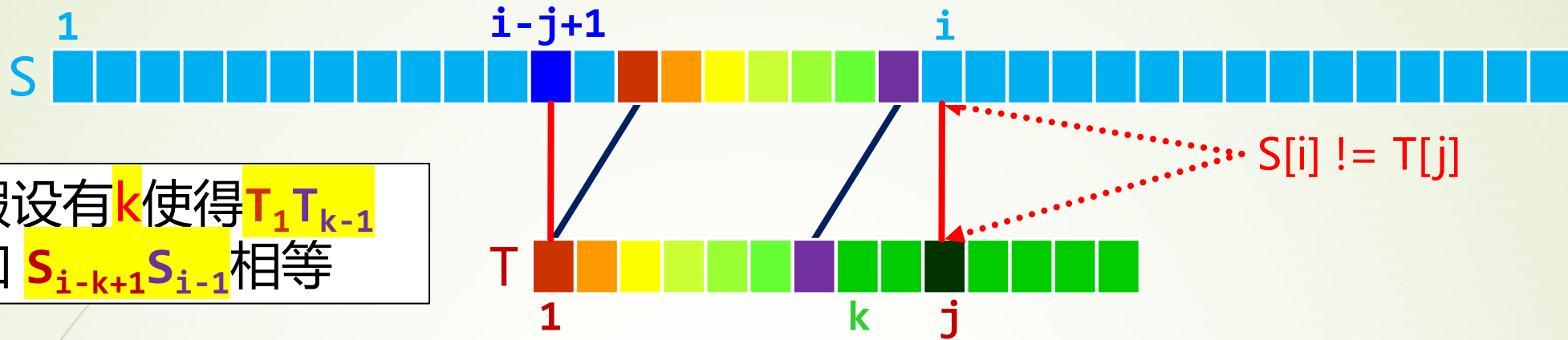
比较次数:1

[illegible]

比较次数:1

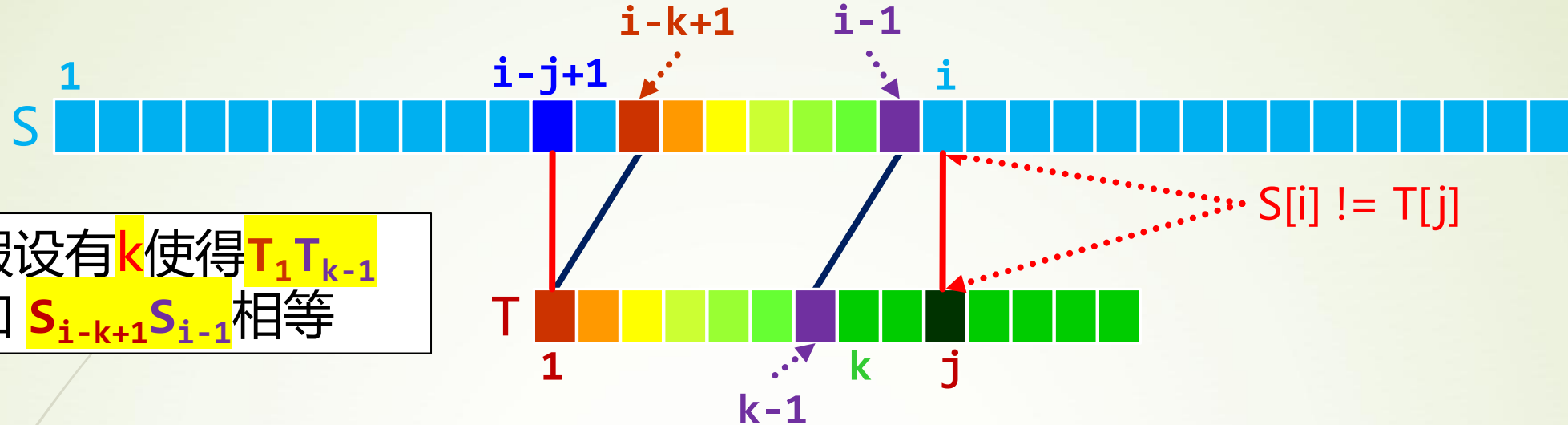
[illegible]



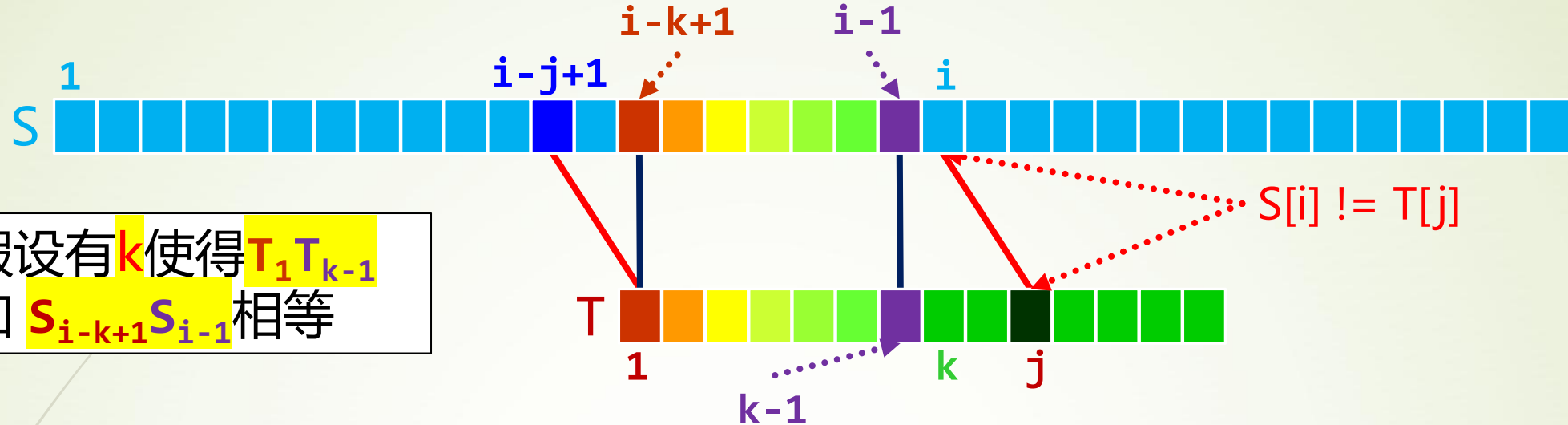


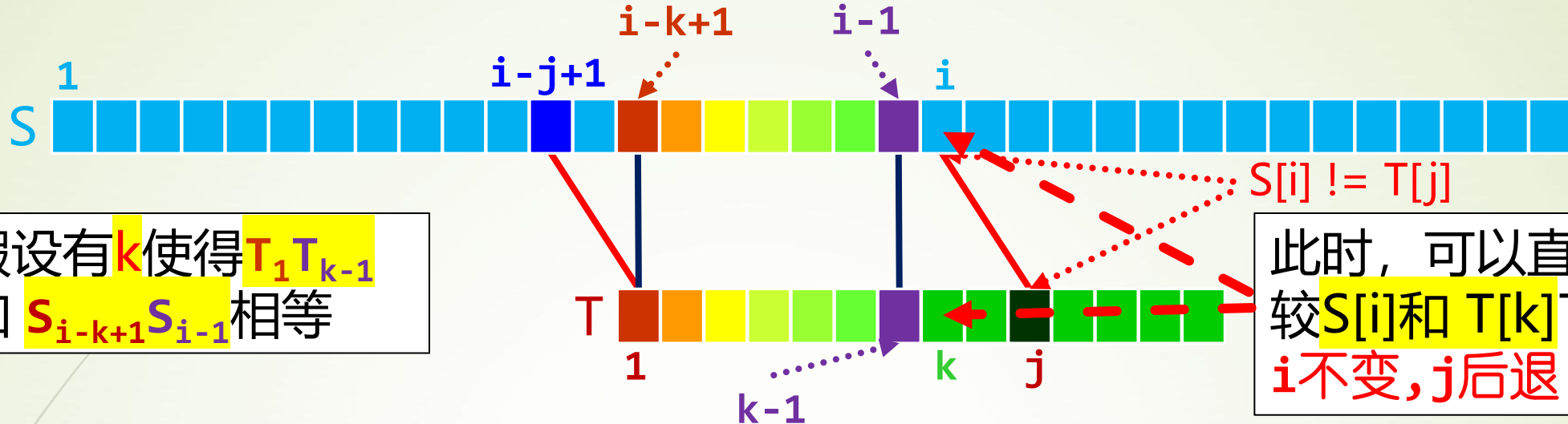
假设有 k 使得 $T_1 T_{k-1}$
和 $S_{i-k+1} S_{i-1}$ 相等

假设有 k 使得 $T_1 T_{k-1}$
和 $S_{i-k+1} S_{i-1}$ 相等



假设有 k 使得 $T_1 T_{k-1}$
和 $S_{i-k+1} S_{i-1}$ 相等





我们引入一个next数组，将j后退时对应的k的值保存在next[j]：

在 $S[i]$ 和 $T[j]$ 失配时($S[i] \neq T[j]$)，可继续对 $S[i]$ 和 $T[\text{next}[j]]$ 进行比较。此时，S串的 i 不需要回溯到 $i-j+2$ 的位置重新开始，而是停在原地不动；当存在 k 时，T串的 j 只需要回溯到next[j]的位置；不存 k 时回溯到1。这样，算法的复杂度降低到 $O(n+m)$

思考一下，next[1]的值是多少？

可以认为是不需要的。因为， $T[1]$ 和 $S[i]$ 不匹配时，T串的 j 无需回溯了。S串的 i 要向后移动一下， $i++$ 。

即，当($S[i] \neq T[j]$)时，如果 $j=1$ ，则 $i++$ ；否则 $j = \text{next}[j]$ 。

BF算法引入next数组进行改造

```
int Index(SString S, SString T,
        int pos)
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j])
        {++i; ++j; }
        else
        {
            i=i-j+2;
            j=1;
        } //i回溯 j回溯
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}
```

修改BF算法的失配回溯部分:
如果 j==1, 则i++, j不变
否则 j=next[j], i不变

```
int Index_1(SString S, SString T,
            int pos)
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j])
        {++i; ++j; }
        else
        {
            if(j==1) i++;
            else j=next[j];
        }
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}
```

next[1]没有用上;
而next[2]到next[T[0]]都有意义, 这稍显别扭。
能不能给next[1]一个值?
给多少?
next[j]表示的是j回退到位置, j==1时已经没有回退的位置了, 所以只能给个0了。
相应的我们修改代码来适应next[1]==0

KMP算法

```
int Index_1(SString S, SString T,
            int pos)
```

```
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j])
        {++i; ++j; }
        else
        {
            if(j==1) i++;
            else j=next[j];
        }
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}
```

规定: next[1] 值为 0,
修改代码适应此值。

```
int Index_KMP(SString S, SString T,
               int pos)
```

```
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j] || j == 0)
        {++i; ++j; }
        else
        {
            j = next[j]; //当j为1时, 将被赋值为0
                        //此时j==0了, 可以和i一起++了
        }
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}
```

```
int Index_KMP (SString S,SString T, int pos)
{
    int i= pos; int j =1;
    while (i <= S[ 0 ] && j <= T [0]) {
        if (j==0 || S[i]==T[j]) {
            i++;j++; }
        else
            j=next[ j ];          /*i不变,j后退*/
    }
    if (j>T[0]) return i-T[0];  /*匹配成功*/
    else return 0;              /*返回不匹配标志*/
}
```

```

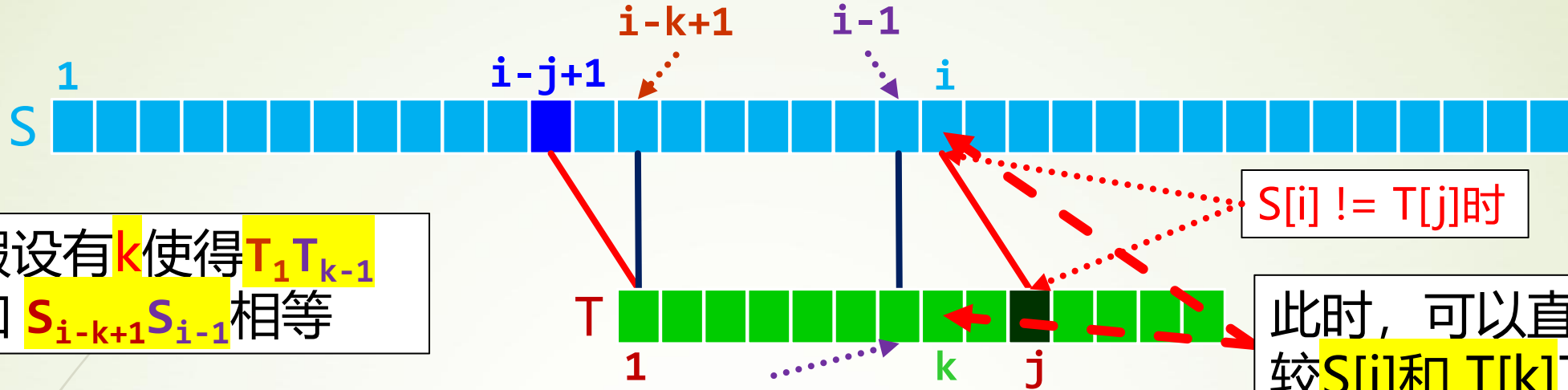
int Index(SString S,
          SString T, int pos)
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j])
        {
            ++i; ++j;
        }
        else
        {
            i=i-j+2; //i回溯
            j=1;      //j回到1
        }
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}

```

```

IntIndex_KMP (SString S,
              SString T,int pos)
{
    int i= pos; int j =1;
    while(i<=S[0] && j<=T[0])
    {
        if(j==0 || S[i]==T[j])
        {
            i++;j++;
        }
        else
        {
            j=next[j]; //i不变 j回退
        }
    }
    if (j>T[0])
        return i-T[0];/*匹配成功*/
    else
        return 0;/*返回不匹配标志*/
}

```



假设有 k 使得 $T_1 T_{k-1}$ 和 $S_{i-k+1} S_{i-1}$ 相等

问题：如何计算出 $next[j]$ ？设 $next[j]$ 为 k

$next[j]$ 为 k ，表示 $S[i-1] == T[k-1] == T[j-1]$ ，
并且串 $T_1 T_{k-1}$ 等于 $S_{i-k+1} S_{i-1}$ 等于 $T_{j-k+1} T_{j-1}$

继续计算 $next[j+1]$

当 $T[j] == T[k]$ 时， $next[j+1] = next[j] + 1$ ；

当 $T[j] != T[k]$ 时，变成一个以 $T[1..k-1]$ 为模式串，以 $T[1..j-1]$ 为主串的模式匹配问题了。

计算next数组算法思路

当有 $\text{next}[j]$ 的值为 k 时:

- (1)如果 $T[j]==T[k]$ 时, $\text{next}[j+1]=\text{next}[j]+1; j++; k++$ 。回到(1)继续。
- (2)如果 $T[j]!=T[k]$ 时, 因1到 j 的next已经都算出了, 按KMP算法, 就可以令 $k=\text{next}[k]$, 回到(1)继续; 即利用KMP的滑动来继续找;
- (3)如果 k 向前滑动后, 一直滑动到 $\text{next}[1]$ 了, 找不到匹配的子串, 此时 $k==0$, 表示需要从 $T[1]$ 开始向后比较, $\text{next}[j+1]=1$; 此时完成 j 的处理, $k=1; j++$; 回到(1)继续。

结束条件:

- 1、当 $j=T[0]$ 时, 计算结束, next已经全部算出。

初始条件:

- 1、 $\text{next}[1] = 0$; //next[1] 不需要计算
- 2、 $j = 1$; //计算next[2]
- 3、 $k?$ 当 $j==1$ 时, k 一定是0

计算next数组算法思路

当有next[j]的值为k时:

- (1)如果 $T[j] == T[k]$ 时,
next[j+1]=next[j]+1;j++;k++。回到(1)继续。
- (2)如果 $T[j] != T[k]$ 时, 因1到j的next已经都算出了, 按KMP算法, 就可以令k=next[k], 回到(1)继续; 即利用KMP的滑动来继续找;
- (3)如果k向前滑动后, 一直滑动到next[1]了, 找不到匹配的子串, 此时k==0, 表示需要从T[1]开始向后比较, next[j+1]=1; 此时完成j的处理, k++; j++; 回到(1)继续。

结束条件:

- 1、当j=T[0]时, 计算结束, next已经全部算出。

初始条件:

- 1、next[1] = 0; //next[1] 不需要计算
- 2、j = 1; //计算next[2]
- 3、k? 当j==1时, k一定是0

计算next数组时, j跟踪主串, k跟踪模式串。

KMP算法种, i跟踪主串, j跟踪模式串。

需要将KMP的 i, j 和 算法思路里的 j, k 对位替换; 并修改初始条件。

```
int Index_KMP (SString S, SString T, int pos)
{
    int i = pos; int j = 1;
    while (i <= S[0] && j <= T[0]) {
        if (j == 0 || S[i] == T[j]) {

            i++; j++;
        }
        else
            j = next[j];          /*i不变, j后退*/
    }
    if (j > T[0]) return i - T[0]; /*匹配成功*/
    else return 0;                /*返回不匹配标志*/
}
```

计算next数组算法思路

当有next[j]的值为k时:

- (1)如果 $T[j] == T[k]$ 时,
next[j+1]=next[j]+1; j++; k++. 回到(1)继续。
- (2)如果 $T[j] != T[k]$ 时, 因1到j的next已经都算出了, 按KMP算法, 就可以令k=next[k], 回到(1)继续; 即利用KMP的滑动来继续找;
- (3)如果k向前滑动后, 一直滑动到next[1]了, 找不到匹配的子串, 此时k==0, 表示需要从T[1]开始向后比较, next[j+1]=1; 此时完成j的处理, k++; j++; 回到(1)继续。

结束条件:

- 1、当j=T[0]时, 计算结束, next已经全部算出。

初始条件:

- 1、next[1] = 0; //next[1] 不需要计算
- 2、j = 1; //计算next[2]
- 3、k? 当j==1时, k一定是0

替换i, j为j, k和, 修改初始化条件后, 按三种情况继续改造...

```
int Index_KMP_N (SString T)
{
    int j=1; next[1]=0; int k =0;
    while ( j <= T[ 0 ] ) {
        if (k==0 || T[j]==T[k]) {
            j++;
            k++;
            next[j]=k; //顺序调整后少算两次加法j+1和k+1
        }
        else
            k=next[ k ]; //和 (2) 一样, 不需要改了
    }
}
```

计算next数组算法思路

当有next[j]的值为k时:

- (1)如果 $T[j] == T[k]$ 时,
next[j+1]=next[j]+1; j++; k++. 回到(1)继续。
- (2)如果 $T[j] != T[k]$ 时, 因1到j的next已经都算出了, 按KMP算法, 就可以令k=next[k], 回到(1)继续; 即利用KMP的滑动来继续找;
- (3)如果k向前滑动后, 一直滑动到next[1]了, 找不到匹配的子串, 此时k==0, 表示需要从T[1]开始向后比较, next[j+1]=1; 此时完成j的处理, k++ ; j++; 回到(1)继续。

结束条件:

- 1、当j=T[0]时, 计算结束, next已经全部算出。

初始条件:

- 1、next[1] = 0; //next[1] 不需要计算
- 2、j = 1; //计算next[2]
- 3、k? 当j==1时, k一定是0

改造完毕

```
void get_next(SString T, int next[])
{
    int j= 1; next[1] = 0; int k = 0;
    while( j<T[0])
    {
        if( k==0 || T[j] == T[k]){
            ++j; ++k;
            next[j] = k;
        }
        else
            k = next[k];
    }
}
```

```

int Index_KMP (SString S,SString T, int pos)
{
    int i= pos; int j =1;
    while (i <= S[ 0 ] && j <= T [0]) {
        if (j==0 || S[i]==T[j]) {
            i++;j++;
        }
        else
            j=next[ j ];          /*i不变,j后退*/
    }
    if (j>T[0]) return i-T[0]; /*匹配成功*/
    else return 0;          /*返回不匹配标志*/
}

```

```

void get_next(SString T, int next[])
{
    i= 1; next[1] = 0; j = 0;
    while( i<T[0]){
        if(j==0 || T[i] == T[j]){
            ++i; ++j;
            next[i] = j;
        }
        else
            j = next[j];
    }
}

```


- KMP算法的时间复杂度

设主串 s 的长度为 n , 模式串 t 长度为 m , 在KMP算法中求 $next$ 数组的时间复杂度为 $O(m)$, 在后面的匹配中因主串 s 的下标不减即不回溯, 比较次数可记为 n , 所以KMP算法总的时间复杂度为 $O(n+m)$ 。

next[j] 数组的函数定义:

表明当模式中第j个字符与主串中相应字符“失配”时，在模式中需重新和主串中该字符进行比较的字符的位置。

$$\text{next}[j] = \begin{cases} \max\{k | 1 < k < j, \{ \text{下一趟比较 } i \text{ 不变, } j=k \} \\ \quad \text{且 "p}_1 \dots \text{p}_{k-1}" = \text{"p}_{j-k+1} \dots \text{p}_{j-1}" \} & \text{当此集合非空时} \\ 0 & \{ \text{代表下一趟比较 } i=i+1, j=1 \} \quad \text{当 } j=1 \text{ 时} \\ 1 & \{ \text{下一趟比较 } i \text{ 不变, } j=1 \} \quad \text{其他情况} \end{cases}$$

```
i = 1; next[1] = 0; j = 0;
while( i < T[0]){
    if(j == 0 || T[i] == T[j]){
        ++i; ++j;
        next[i] = j;
    }
    else // T[i] != T[j]
        j = next[j];
}
```

[illegible]

next[j] 数组的函数定义:

表明当模式中第j个字符与主串中相应字符“失配”时，在模式中需重新和主串中该字符进行比较的字符的位置。

$\text{next}[j] = \begin{cases} \max\{k | 1 < k < j, \{ \text{下一趟比较 } i \text{ 不变, } j=k \} \\ \text{且 } "p_1 \dots p_{k-1}" = "p_{j-k+1} \dots p_{j-1}" \} & \text{当此集合非空时} \\ 0 & \{ \text{代表下一趟比较 } i=i+1, j=1 \} \quad \text{当 } j=1 \text{ 时} \\ 1 & \{ \text{下一趟比较 } i \text{ 不变, } j=1 \} \quad \text{其他情况} \end{cases}$

```

i = 1; next[1] = 0; j = 0;
while( i < T[0] ){
    if( j == 0 || T[i] == T[j] ){
        ++i; ++j;
        next[i] = j;
    }
    else // T[i] != T[j]
        j = next[j];
}
    
```

j	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
模式串	a	b	c	a	a	b	b	c	a	b	c	a	a	b	d	a	b
next[j]	0	1	1	1	2	2	3	1	1	2	3	4	5	6	7	1	2

循环次数	1
i	1
j	0
T[i]	
T[j]	
next[j]	

● next函数的改进

①

$\downarrow i=4$
 S: a a a b a a a a b
 T: a a a a b
 $\uparrow j=4$ next[4] == 3
 T: a a a a b
 $\uparrow T[next[3]] == T[j]$
 T: a a a a b
 $\uparrow T[next[2]] == T[j]$
 T: a a a a b
 $\uparrow T[next[1]] == T[j]$
 T: a a a a b
 $i = 5; j = 1$

②

j	1	2	3	4	5
模式	a	a	a	a	b
next[j]	0	1	2	3	4
nextval[j]	0	0	0	0	4

KMP算法在失配时 $S[i] \neq T[j]$ ，通过next[j]将T[j]滑动到T[next[j]]进行再次匹配尝试。

此时若发现，T[j]、T[next[j]]、T[next[next[j]]]...等等都相同，就可以一步到位滑动到和T[j]不同的T[next[next[...]]]位置。

这是在S[i]和T[j]失配时进行的，显然用T[j]去比较和发现最终位置，和KMT算法执行时用S[i]去比较滑动是一回事，并没有效率提升。

但是T[j]和T[next[next[...]]]都是模式串T中的，能否在生成next数组时根据这个发现来改进呢？

- next函数的改进：将在KMP中的T向前比较改为在生成next值时进行，由于生成时从前向后生成，在生成i位置的next值时，只需向前比较一次即可。
- get_next算法改进为get_nextval数组，数组名改为nextval

```
void get_next(SString T, int next[])
```

```
{
```

```
    i= 1; next[1] = 0; j = 0;
```

```
    while( i<T[0]){
```

```
        if(j==0 || T[i] == T[j]){
```

```
            ++i; ++j;
```

```
            next[i] = j;
```

```
        }
```

```
    else
```

```
        j = next[j];
```

```
    }
```

```
}
```

```
void get_nextval(SString T, int nextval[])
```

```
{
```

```
    i= 1; nextval[1] = 0; j = 0;
```

```
    while( i<T[0]){
```

```
        if(j==0 || T[i] == T[j]){
```

```
            ++i; ++j;
```

```
            if(T[i] != T[j]) nextval[i] = j;
```

```
            else nextval[i] = nextval[j];
```

```
        }
```

```
    else
```

```
        j = nextval[j];
```

```
    }
```

```
}
```

```
if(T[i] != T[j]) nextval[i] = j;  
else nextval[i] = nextval[j];
```

j	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
模式串	a	b	c	a	a	b	b	c	a	b	c	a	a	b	d	a	b
next[j]	0	1	1	1	2	2	3	1	1	2	3	4	5	6	7	1	2
nextval[j]	0	1	1	0	2	1	3	1	0	1	1	0	2	1	7	0	1

```

int Index(SString S,
          SString T, int pos)
{
    int j=1, i=pos;
    while(i<=S[0] && j<= T[0])
    {
        if( S[i]==T[j])
        {
            ++i; ++j;

        }
        else
        {
            i=i-j+2;
            j=1;
        } //i回溯 j回溯
    }
    if( j > T[0] )
        return i - T[0];
    else
        return 0;
}

```

```

IntIndex_KMP (SString S,
              SString T,int pos)
{
    int i= pos; int j =1;
    while(i<=S[0] && j<=T[0])
    {
        if(j==0 || S[i]==T[j])
        {
            i++;j++;

        }
        else
        {
            j=next[j];

        }
    }
    if (j>T[0])
        return i-T[0];/*匹配成功*/
    else
        return 0;/*返回不匹配标志*/
}

```

```

void get_nextval(SString T,
                 int &nextval[])
{
    int i=1,j=0; nextval[1]=0;
    while( i<T[0])
    {
        if(j==0 || T[i] == T[j])
        {
            ++i; ++j;
            if(T[i]!= T[j])
                nextval[i] = j;
            else
                nextval[i]=nextval[j];
        }
        else
        {
            j = nextval[j];
        }
    }
}

```

一些字符串匹配算法

► 1、Boyer-Moore算法

在用于查找子字符串的算法当中，BM (Boyer-Moore) 算法被认为最高效的字符串搜索算法，它由Bob Boyer和J Strother Moore设计于1977年。BM算法在移动模式串的时候是从左到右，而进行比较的时候是从右到左的。BM算法实际上包含两个并行的算法（也就是两个启发策略）：坏字符算法 (bad-character shift) 和好后缀算法 (good-suffix shift)。这两种算法的目的就是让模式串每次向右移动尽可能大的距离。

► 2、Horspool算法

Boyer-Moore-Horspool 算法也称Horspool 算法，由Nigel Horspool设计于1980年，是在BM算法上改进版，因为BM算法里面的 好后缀规则较难理解，同时其效率与正确性的证明当时一直没有得到解决，所以Horspool 算法只用了一个BM里的坏字符规则。Horspool算法也是从右向左比较，但Horspool算法相对于Boyer-Moore算法改进了坏字符规则。

► 3、Sunday算法

Sunday算法由Daniel M.Sunday在1990年提出，它的思想跟BM算法很相似；只不过Sunday算法是从前往后匹配，在匹配失败时关注的是主串中参加匹配的最末位字符的下一位字符。

► 4、KR算法

Karp Rabin 算法是利用hash函数的特性进行字符串匹配的。KR算法对模式串和循环中每一次要匹配的子串按一定的hash函数值，如果hash值相同，才运用BF算法匹配。如果hash函数设计得好的话，不同子串hash值相同是小概率事件，且速度较快。在虽然理论上KR算法的时间复杂度是 $O(m*n)$ ，但现实应用中一般是 $O(m+n)$ （模式串在主串中出现的次数较少时适合此算法）。

► 5、AC自动机

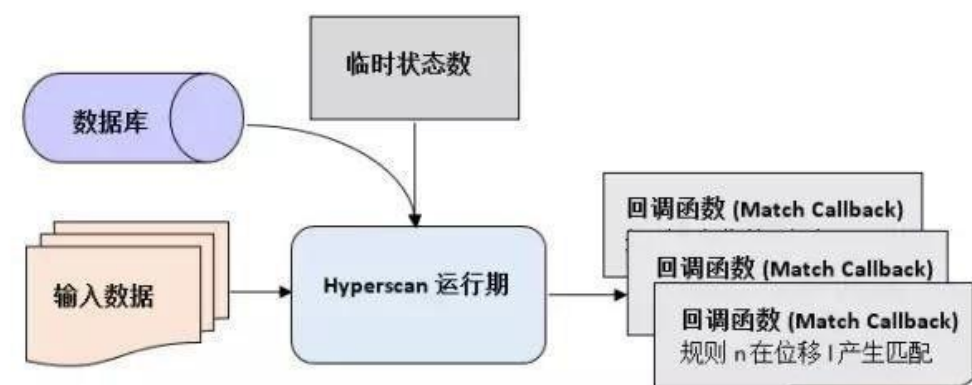
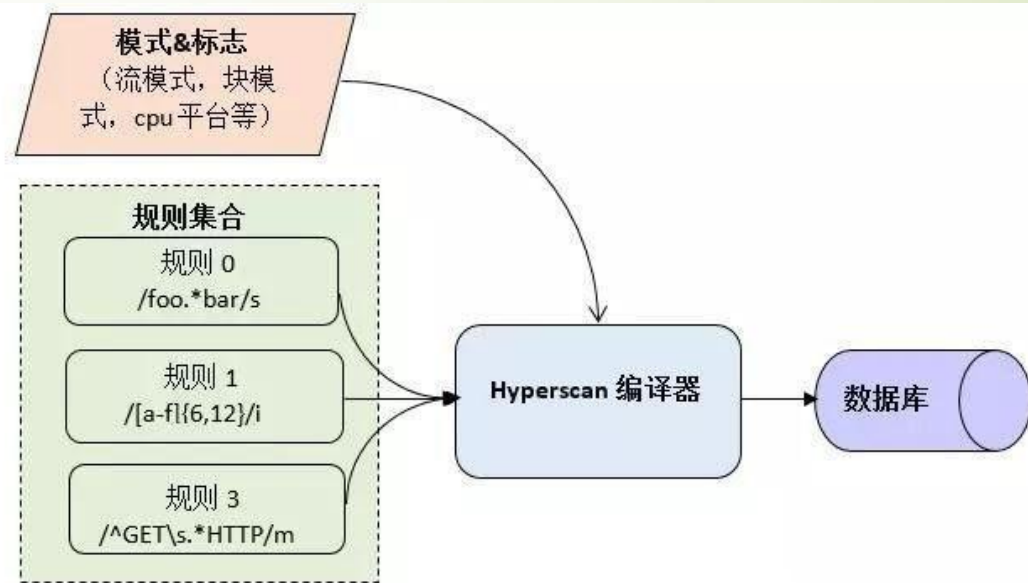
最著名的多模式匹配算法为AC自动机，它是由贝尔实验室的两位研究人员 Alfred V. Aho 和 Margaret J. Corasick 于1975年发明的，几乎与KMP算法同时问世，至今日仍然在模式匹配领域被广泛应用。在多模式环境中，AC自动机是使用前缀树来存放所有模式串的前缀，然后通过失配指针来处理失配的情况。它大概分为三个步骤：构建前缀树（生成goto表），添加失配指针（生成fail表），模式匹配（构造output表）。

基于正则表达式的串模式匹配算法

- 正则表达式, **Regular Expression**, 如`*.ppt?`, 使用单个字符串描述一系列满足某种句法规则的字符串作为模式串。
- 在很多文本编辑器里, 正则表达式通常被用来检索、替换那些匹配某个模式的文本。
- 正则表达式这个概念最初是由Unix中的工具软件(例如**sed**和**grep**)普及开的。
- **Ken Thompson**就把这一成果应用于计算搜索算法的一些早期研究, Ken Thompson是 Unix的主要发明人, 也就是大名鼎鼎的Unix之父。Unix之父将此符号系统引入编辑器**QED**, 然后是Unix上的编辑器**ed**, 并最终引入**grep**。**Jeffrey Friedl** 在其著作《**Mastering Regular Expressions (2nd edition)**》(中文版译作: 精通正则表达式, 已出到第三版)中对此作了进一步阐述讲解, 如果你希望更多了解正则表达式理论和历史, 推荐你看看这本书。
- 在C/C++中常用的正则表达式库有**GNU Regex Library**, **Boost.Regex**, **PCRE**, **PCRE++**等
- 一个正则表达式引擎的设计和实现1-汤普森构造
- https://blog.csdn.net/tyler_download/article/details/53033180
- 代码实现Thompson构造: 由简单到复杂的构建**NFA**状态机
- https://blog.csdn.net/tyler_download/article/details/51382346

基于正则表达式的串模式匹配算法的高效实现

- Hyperscan是一款来自于Intel的高性能的正则表达式匹配库。它是基于X86平台以PCRE为原型而开发的，并以BSD许可开源在<https://01.org/hyperscan>。
- 在支持PCRE的大部分语法的前提下，Hyperscan增加了特定的语法和工作模式来保证其在真实网络场景下的实用性。与此同时，大量高效算法及Intel SIMD*指令的使用实现了Hyperscan的高性能匹配。
- Hyperscan以自动机理论为基础，其工作流程主要分成两个部分：编译期(compiletime)和运行期(run-time)。
- <https://zhuanlan.zhihu.com/p/40603962>





1. 熟悉串的基本操作的定义，并能利用这些基本操作来实现串的其它各种操作。

2. 了解串的各种存储结构的特点及其适用场合。

3. 理解串的两模式匹配算法。

4.4 串应用示例——文本编辑

[分析]

- 操作对象
——文本串
(行是文本串的子串)
- 基本操作
 - 查找
 - 插入
 - 删除

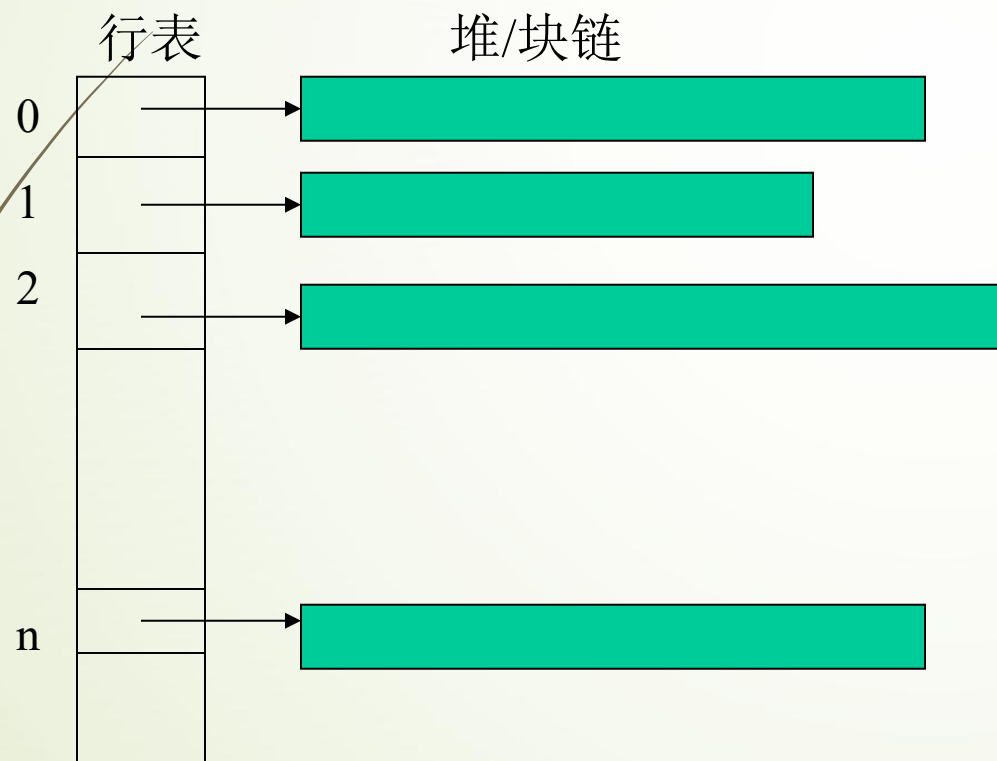
```
#include <stdio.h>
int main (void)
{
    printf("Hello world!\n");
    return 0;
}
```

[存储结构选择]

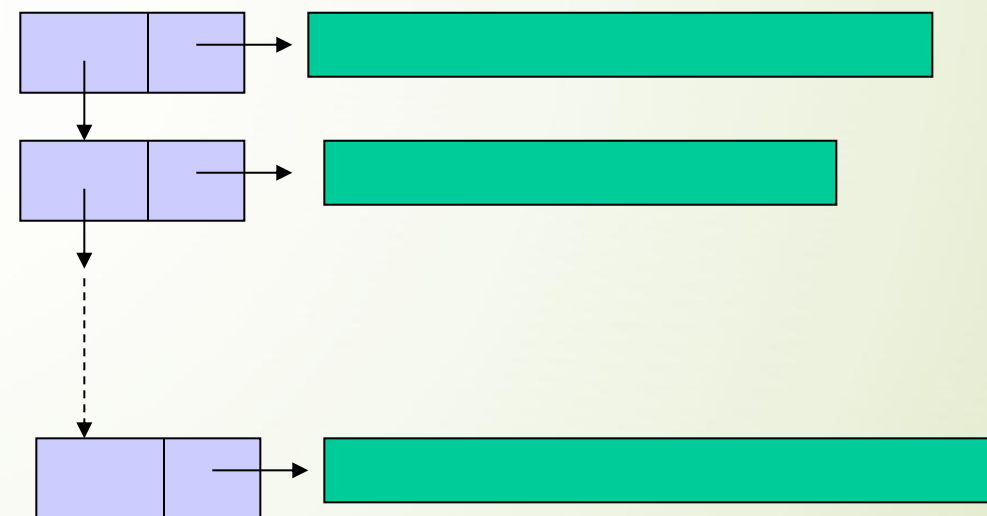
方案一 简单顺序存储



方案二



• 方案三



思考：

- 1) 三个方案的优缺点各是什么？从初始化（加载磁盘文件到内存）、在电脑窗口显示内容、插入文字、删除文字、查找文字、替换文字、操作结束后存盘这几个功能进行时间复杂度的比较。对每种数据结构的某个时间复杂度大的功能，如何采用一些技巧来改进？
- 2) 电脑窗口显示的内容和全部内容之间的关系？如何利用这个部分和整体的关系来加速内容显示？会带来什么问题？
- 3) 存在磁盘上的文件大小和加载到内存中的内容大小之间的关系？如何利用这个关系，在超大文件操作方面，能加速哪些功能？会带来什么问题？
- 4) 找找看，实际的软件采用的是什麼数据结构？找开源软件读读文档？找关键代码读读代码？